Lux: Technical Architecture Specification

Visual Output Surface for AI Agents

April 2026 — v0.16.1

Contents

1 Overview **3**
- 1.1 Design Principles . . . 3

2 System Architecture **3**
- 2.1 Component Map . . . 3
- 2.2 Data Flow . . . 4
  - 2.2.1 Write Path (Agent → Display) . . . 4
  - 2.2.2 Event Path (User → Agent) . . . 4
  - 2.2.3 Update Path (Incremental Patching) . . . 4

3 Wire Protocol **4**
- 3.1 Framing . . . 4
- 3.2 Message Types . . . 5
  - 3.2.1 Client → Display . . . 5
  - 3.2.2 Display → Client . . . 5
- 3.3 Element Kinds . . . 5

4 Display Server Internals **6**
- 4.1 Frame Cycle . . . 6
- 4.2 Multi-Scene State . . . 7
  - 4.2.1 Scene Lifecycle . . . 7
  - 4.2.2 Widget State Swap Pattern . . . 7
- 4.3 Table Filtering . . . 8

5 Frame Architecture **8**
- 5.1 _Frame Dataclass . . . 8
- 5.2 Frame Lifecycle . . . 8
- 5.3 Dock Bar . . . 9
- 5.4 Frame Rendering . . . 9

6 World Menu and Applications **9**
- 6.1 Applications Menu . . . 9
- 6.2 Per-Client Menu Namespaces . . . 9

7 IPC and Process Management **9**
- 7.1 Socket Discovery . . . 9
- 7.2 PID Lifecycle . . . 10
- 7.3 Auto-Spawn . . . 10
- 7.4 MCP Server Reconnect . . . 10

8 Performance **10**
- 8.1 Frame Budget . . . 10
- 8.2 Critical Path Analysis . . . 10
- 8.3 Idle Screen . . . 10
- 8.4 Memory . . . 11
- 8.5 Known Limitations . . . 11

9 Security **11**
9.1 Threat Model 11
9.2 Render Function Security 11
9.3 Protocol Robustness 12

10 Platform Integration **12**
10.1 macOS 12
10.2 Linux 12

11 MCP Tool Surface **13**

12 Formal Model **13**

13 Test Architecture **14**

1 Overview

Lux is a visual output surface for Claude Code. It renders element trees pushed by AI agents via MCP tools into a native ImGui window over Unix socket IPC. The system has four projection surfaces from a single codebase: Python library, CLI, MCP server, and Claude Code plugin.

The display server owns the render loop. Agents are clients. All agent–display communication is message-passing over a framed JSON protocol on a Unix domain socket. The display server is single-threaded: no threads, no asyncio, no locks. It polls all I/O non-blocking each frame via `select()`.

1.1 Design Principles

P1. **Composable element tree.** Every element kind is small, nestable, and data-driven. If an agent can describe it as JSON, Lux renders it.

P2. **Protocol is the API surface.** The wire protocol is the only interface between agents and the display. No shared memory, no function calls, no Python imports required.

P3. **Single-threaded simplicity.** The display server uses a polling event loop with zero threads. All state is owned by the main thread. No synchronization primitives.

P4. **Incremental by default.** Scenes can be patched by element ID without replacing the entire tree. Table filters run at frame rate with zero MCP round-trips.

P5. **Multi-scene coexistence.** Each `show()` call opens a new tab. Scenes persist until explicitly dismissed or cleared. Same `scene_id` replaces content in-place.

2 System Architecture

2.1 Component Map

Module	File	Responsibility
Protocol	`src/punt_lux/protocol.py`	Message types, element dataclasses, wire format (framing, serialization, deserialization)
Display	`src/punt_lux/display.py`	ImGui render loop, socket IPC, scene state, widget state, element renderers, event flush
Client	`src/punt_lux/client.py`	`LuxClient` context manager; connects, sends, receives, buffers pending messages
MCP Server	`src/punt_lux/server.py`	FastMCP tools (`show`, `update`, `clear`, `ping`, `recv`, convenience wrappers); auto-reconnect
Paths	`src/punt_lux/paths.py`	Socket discovery, PID lifecycle, auto-spawn
Consent	`src/punt_lux/consent.py`	ImGui modal for render function approval
Runtime	`src/punt_lux/runtime.py`	`CodeExecutor`; compiles and runs render functions per-frame
AST Check	`src/punt_lux/ast_check.py`	AST visitor for suspicious imports and builtins (UX signal, not security boundary)
Config	`src/punt_lux/config.py`	`.lux/config.md` YAML frontmatter; display mode toggle
Hooks	`src/punt_lux/hooks.py`	Claude Code lifecycle hook handlers
CLI	`src/punt_lux/__main__.py`	Typer CLI: display, serve, enable, disable, status, doctor, install, uninstall
Show	`src/punt_lux/show.py`	CLI subcommands for pre-built scenes (beads board)
Apps/Beads	`src/punt_lux/apps/beads.py`	Beads issue browser; loads `.beads/issues.jsonl`, renders filterable table
Apps/Calc	`src/punt_lux/apps/calculator.py`	Programmer calculator; multi-base display with bit grid and bitwise operations
Apps/Clock	`src/punt_lux/apps/clock.py`	Analog clock; smooth hand animation via render function

2.2 Data Flow

Three primary data paths connect agents to the display:

2.2.1 Write Path (Agent → Display)

1. Agent calls MCP tool (e.g., `show()`).
2. MCP server creates `LuxClient`, calls `client.show(scene_id, elements)`.
3. Client serializes `SceneMessage` as JSON, frames with 4-byte big-endian length prefix, sends over Unix socket.
4. Display receives frame in `_poll_clients()`, deserializes, stores in `_scenes` dict.
5. Display renders scene each frame via `_render_scene()`.
6. Display sends `AckMessage` back to client.

2.2.2 Event Path (User → Agent)

1. User interacts with widget (click, slider drag, text input).
2. Display generates `InteractionMessage` with `element_id`, `action`, and `value`.
3. Message buffered in `_event_queue`.
4. Each frame, `_flush_events()` broadcasts all queued messages to all connected clients, then clears the queue.
5. Agent calls `recv()` to consume events.

2.2.3 Update Path (Incremental Patching)

1. Agent calls `update(scene_id, patches)`.
2. Display receives `UpdateMessage`, calls `_apply_update()`.
3. `_find_element()` locates target by ID via depth-first search through the element tree.
4. Patch applied: `set` updates fields in-place, `remove` deletes element and cleans up widget state.
5. Routes to correct scene by `scene_id` (not limited to active tab).

3 Wire Protocol

3.1 Framing

Every message is a length-prefixed JSON frame:

```
+-----------------+---------------------------+
| 4 bytes (uint32) | N bytes (UTF-8 JSON) |
| big-endian | payload |
+-----------------+---------------------------+
```

Maximum payload size: 16 MiB (2^{24} bytes). The `FrameReader` accumulates partial reads into a byte buffer and yields complete frames. Oversize frames trigger client disconnection.

3.2 Message Types

3.2.1 Client → Display

Type	Purpose
`SceneMessage`	Replace or add scene. Fields: `id`, `elements[]`, `title`, `layout`.
`UpdateMessage`	Incremental patches. Fields: `scene_id`, `patches[]` (each: `id`, `set`, `remove`).
`ClearMessage`	Remove all scenes and reset state.
`PingMessage`	Heartbeat. Field: `ts`.
`MenuMessage`	Set agent-provided menu entries.
`ThemeMessage`	Apply theme by name.
`ConnectMessage`	Client identity handshake. Field: `name`.
`RegisterMenuMessage`	Register per-client menu items. Field: `items[]`.

SceneMessage frame fields (since v0.9.0): `SceneMessage` includes optional fields `frame_id`, `frame_title`, `frame_size` (initial `[width, height]`), `frame_flags` (dict of ImGui window flags such as `no_resize`, `no_collapse`, `no_title_bar`, `no_background`, `no_scrollbar`, `auto_resize`), and `frame_layout` (`"tab"` or `"stack"`).

3.2.2 Display → Client

Type	Purpose
`ReadyMessage`	Handshake on connect; protocol version, capabilities.
`AckMessage`	Acknowledges scene or update; `scene_id`, `ts`, optional `error`.
`InteractionMessage`	User interaction; `element_id`, `action`, `value`, `ts`.
`WindowMessage`	Window lifecycle (resized, closed, focused).
`PongMessage`	Ping response; round-trip latency probe.

3.3 Element Kinds

27 element kinds organized by category:

Category	Element	Notes
Text	`text`	Content + style (heading, caption, success, error, code); optional `color` hex string
	`markdown`	CommonMark rendering via imgui_md
Media	`image`	File path or base64 data
Interactive	`button`	Label, action, disabled flag; `arrow` (directional) and `small` variants
	`slider`	Min/max/value, integer mode
	`checkbox`	Boolean toggle
	`combo`	Dropdown selection
	`input_text`	Single-line text input
	`radio`	Radio button group
	`input_number`	Numeric input with step buttons, min/max clamping, integer mode
	`color_picker`	RGB/RGBA hex picker; `alpha` (RGBA) and `picker` (full widget) modes
Data	`table`	Columns, rows, built-in filters, detail panel, pagination

Category	Element	Notes
	`plot`	ImPlot: line, scatter, bar series
	`progress`	Fraction bar with label
	`spinner`	Loading indicator
Lists	`selectable`	Click-to-select item
	`tree`	Recursive node hierarchy; `flat` flag for non-indented expansion
Drawing	`draw`	Canvas with draw commands
	`render_function`	Agent-submitted Python (consent-gated)
Layout	`modal`	Modal popup dialog; container with children, emits `"closed"` event
	`group`	Row, column, or paged layout (`pages` + `page_source`)
	`tab_bar`	Tabbed container
	`collapsing_header`	Expandable section
	`window`	Movable, resizable floating panel
	`separator`	Visual divider

4 Display Server Internals

4.1 Frame Cycle

The display server runs an ImGui render loop via `immapp.run()`. Each frame executes four phases in sequence:

1. **Accept connections.** `select()` on server socket with timeout 0. Accept if readable. Send `ReadyMessage` to new client.

2. **Poll clients.** `select()` on all client sockets with timeout 0. Read available data into per-client `FrameReader`. Dispatch complete messages to `_handle_message()`. Remove errored sockets.

3. **Render scene.** Enable docking via `dock_space_over_viewport`. Render idle flame background. Check for World menu background click. Render World panel if open. Render all workspace frames (see §5). Render dock bar for minimized frames. Legacy fallback: if no frames, render unframed scenes as tabs.

4. **Flush events.** Broadcast all buffered `InteractionMessage`s to all connected clients, then clear the queue.

All four phases are non-blocking. The frame completes in microseconds when idle. ImGui handles frame pacing via GLFW vsync.

4.2 Multi-Scene State

Field	Type and Purpose
_scenes	dict[str, SceneMessage] — ordered by insertion; keyed by scene_id.
_scene_order	list[str] — explicit tab ordering.
_active_tab	str | None — currently selected tab.
_scene_widget_state	dict[str, WidgetState] — per-scene slider/checkbox/combo/text state.
_scene_render_fn_state	dict[str, dict[str, _RenderFnState]] — per-scene render function lifecycle.
_event_queue	list[InteractionMessage] — global, shared across all scenes.
_dirty_windows	set[str] — window element IDs needing initial position set.
Frame state (since v0.9.0)	
_frames	dict[str, _Frame] — all workspace frames, keyed by frame_id.
_focus_frame_id	str | None — frame to auto-focus on next render.
_scene_to_frame	dict[str, str] — scene → frame lookup.
_scene_to_owner	dict[str, int] — scene → client fd ownership.
Client identity (since v0.9.0)	
_client_names	dict[int, str] — client fd → display name.
_menu_registrations	dict[int, list] — per-client menu items.
_menu_owners	dict[str, int] — menu item ID → owner fd.

4.2.1 Scene Lifecycle

New scene (scene_id not in _scenes)
Appended to _scenes and _scene_order. Fresh WidgetState and render function state allocated. Becomes the active tab. Window elements marked dirty. Existing events preserved.

Replace-in-place (same scene_id)
Content replaced in _scenes. Widget state and render function state cleared for that scene. Events for elements that existed in the old scene but not the new scene are drained (using full subtree ID collection via _collect_ids()). Events from other scenes are untouched.

Dismiss (tab close button)
Scene removed from all state dicts. Window element IDs removed from _dirty_windows. Active tab moves to the adjacent neighbor (next tab, or previous if rightmost was dismissed).

Clear (ClearMessage or menu)
All scenes, widget state, render function state, events, and dirty windows cleared. Fresh instances allocated (not .clear() on aliased objects).

4.2.2 Widget State Swap Pattern

The 20+ element renderers all read from self._widget_state and self._render_fn_state. Rather than threading a scene_id parameter through every method, the display swaps the per-scene state dicts into these instance variables before rendering each tab:

```
def _render_scene_tab(self, scene_id: str) -> None:
    self._widget_state = self._scene_widget_state[scene_id]
    self._render_fn_state = self._scene_render_fn_state[scene_id]
    scene = self._scenes[scene_id]
    for elem in scene.elements:
        self._render_element(elem)
```

This is the same pattern ImGui itself uses: "current context." Zero changes required in any renderer method.

4.3 Table Filtering

Tables support built-in filtering at frame rate with zero MCP round-trips. The display server renders filter controls (search input, combo dropdowns) above the table and applies filters in the render loop:

1. Filter state stored in `WidgetState` with prefixed keys (e.g., `__tbl_search_{eid}`, `__tbl_filter_{col}_{eid}`).
2. `_apply_table_filters()` renders controls, snapshots state, detects changes.
3. `_filter_indexed_rows()` applies AND logic across all active filters (case-insensitive substring for search, exact match for combos).
4. Pagination resets to page 0 when filters change.
5. Detail panel renders a two-column metadata grid plus long-form body text for the selected row.

5 Frame Architecture

Since v0.9.0, scenes can target named *frames*—persistent ImGui windows that outlive client connections. Frames are the primary unit of workspace organization, following the Smalltalk Morphic pattern of composable, persistent windows.

5.1 _Frame Dataclass

Field	Type and Purpose
`frame_id`	`str` — Unique identifier.
`title`	`str` — Window title bar text.
`owner_fds`	`set[int]` — File descriptors of contributing clients.
`scenes`	`dict[str, SceneMessage]` — All scenes in this frame, keyed by `scene_id`.
`scene_order`	`list[str]` — Tab ordering.
`active_tab`	`str | None` — Currently visible scene.
`minimized`	`bool` — Docked to bottom bar.
`cascade_index`	`int` — Stagger offset for initial positioning.
`initial_size`	`tuple[int,int] | None` — Width/height hint (first use only).
`flags`	`dict[str,bool] | None` — ImGui window flags (`no_resize`, `no_collapse`, `no_title_bar`, `no_background`, `no_scrollbar`, `auto_resize`).
`layout`	`"tab" | "stack"` — Multi-scene composition mode.

5.2 Frame Lifecycle

Creation

A frame is created on the first `SceneMessage` with a `frame_id` not already in `_frames`. The client's fd is added to `owner_fds`.

Shared ownership

Multiple clients can contribute scenes to the same frame. Each client's fd is added to `owner_fds` on scene insertion.

Client disconnect

On disconnect, the client's fd is removed from `owner_fds`. Scenes owned by the departing client are re-assigned to a remaining owner if one exists, or marked as orphans (`_ORPHAN_FD = -1` sentinel). Frames persist with orphaned scenes—they are never dismissed on disconnect.

Focus

Every `SceneMessage` targeting a frame sets `_focus_frame_id`. The next render pass calls `imgui.set_next_window_fo` on that frame, then clears the field. Minimized frames are restored on focus.

Close

Frames are removed only by: user clicking the X button, `Clear All` in the Windows menu, or a `ClearMessage` from any client.

5.3 Dock Bar

Minimized frames appear as clickable pills in a persistent bar at the bottom of the viewport. Click a pill to restore and focus the frame. The bar is only rendered when at least one frame is minimized.

The Windows menu provides bulk operations: `Collapse All` (minimize all visible frames), `Expand All` (restore all minimized frames), and `Fit All` (tile frames in a non-overlapping grid layout).

5.4 Frame Rendering

Within `_render_frames()`, each render pass:

1. Computes default frame size (75% of content region).
2. Applies fit-all tiling layout if active.
3. For each non-minimized frame: renders an ImGui window via `_render_single_frame()`.
4. Dispatches frame contents based on `layout`: `"tab"` renders a closable tab bar; `"stack"` renders vertically stacked collapsing headers.
5. Renders the dock bar with minimized frame pills.

6 World Menu and Applications

The display server provides a floating World menu panel (inspired by Pharo Smalltalk's World menu) triggered by clicking the viewport background. The panel mirrors the menu bar structure: Lux, Applications, Debug, Windows, Help. It supports pin/unpin; when unpinned, it auto-closes on item click.

6.1 Applications Menu

Built-in applets register in the Applications menu via `declare_menu_item()` and `on_event()` callbacks:

- **Beads Browser** — opens the beads issue board in a frame. Reads `.beads/issues.jsonl` from the MCP server's working directory.
- **Programmer Calculator** — multi-base integer calculator with bit grid and bitwise operations.
- **Analog Clock** — smooth-sweeping clock face rendered via ImGui draw list in a chromeless floating window.

6.2 Per-Client Menu Namespaces

Each connected MCP server registers menu items via `RegisterMenuMessage`. Items are grouped by client name in the Applications menu. Item clicks generate `InteractionMessage` events routed only to the owning client (not broadcast). Menu items and ownership are cleaned up on client disconnect.

7 IPC and Process Management

7.1 Socket Discovery

Socket path resolution, in priority order:

1. `$LUX_SOCKET` environment variable.
2. `$XDG_RUNTIME_DIR/lux/display.sock`.
3. `/tmp/lux-$USER/display.sock` (fallback).

The fallback path is chosen to stay within macOS's ~104-character `AF_UNIX` path limit.

7.2 PID Lifecycle

- PID file written at {`socket_path`}`.pid` on server startup.
- Clients verify liveness via `os.kill(pid, 0)` before connecting.
- Stale socket cleanup: `cleanup_stale_socket()` removes socket and PID file if the process is dead.

7.3 Auto-Spawn

`ensure_display()` spawns the display server if not running:

1. Checks socket existence and PID liveness.
2. If not running, spawns
   `python -m punt_lux display --socket {path}`
   with `start_new_session=True` (detached from caller's process group).
3. Polls socket existence + PID liveness every 100 ms until ready or 5 s timeout.
4. Display logs to {`socket_path`}`.log`.

7.4 MCP Server Reconnect

The MCP server wraps every tool call in `_with_reconnect()`: if an `OSError` (broken pipe) occurs, the stale client is closed, a new connection is established, and the call is retried exactly once. No retry loop.

8 Performance

8.1 Frame Budget

Parameter	Value	Notes
Idle FPS	30	`fps_idling.fps_idle = 30.0`
Active FPS	60+	GLFW vsync, GPU-bound. Docking enabled via `DockingEnable` config flag (v0.12.0).
Socket poll timeout	0	Non-blocking `select()`
Max message size	16 MiB	Hard limit in `FrameReader`
Connection timeout	5.0 s	`ensure_display()` and handshake
Texture loading	Lazy	First render triggers PIL + OpenGL upload

8.2 Critical Path Analysis

The frame-critical path is:

1. `select()` on N client sockets — $O(N)$, $N < 64$ in practice.
2. `FrameReader.feed()` — memcpy into buffer, yield frames. $O(B)$ where B is bytes received.
3. `_render_scene()` — ImGui draw list calls. $O(E)$ where E is total element count across visible scene.
4. `_flush_events()` — send K events to N clients. $O(K \cdot N)$.

Table filtering is $O(R \cdot C)$ where R is row count and C is column count. This runs every frame but is fast for tables under ∼10,000 rows. No benchmarks exist for larger tables; pagination (configurable page size) bounds visible row rendering.

8.3 Idle Screen

The idle screen renders ∼60 draw calls per frame: 48 radial lines, 3 glow circles, 3 flame shapes (each ∼7 bezier operations). This is negligible for a GPU-rendered immediate-mode UI at 30 FPS idle.

8.4 Memory

- **Scene storage.** Scenes are Python dataclass trees held in memory. No size limit beyond the 16 MiB message cap. Each scene is a separate dict entry; dismissed scenes are garbage-collected.
- **Texture cache.** OpenGL texture IDs held until `_on_exit()`. No eviction policy. Image-heavy scenes accumulate GPU memory.
- **Widget state.** Per-scene `WidgetState` is a flat dict. Cleared on scene replacement. No growth concern.
- **Event queue.** Flushed every frame. Bounded by interaction rate (<100 events/frame in practice).

8.5 Known Limitations

1. **No event attribution.** `InteractionMessage` carries `element_id` but not `scene_id`. If two scenes use the same element ID, events are ambiguous. (Tracked: lux-308.)
2. **No ImGui ID scoping.** Widget IDs are derived from `element_id` alone, not scoped by scene. ImGui internal state (focus, open/closed) can bleed between tabs with colliding IDs.
3. **No texture eviction.** Long-running sessions with many images accumulate GPU memory.
4. **Filter is per-frame.** Table filtering re-executes every frame even when nothing changed. Not a problem at current scale but would benefit from dirty-flag optimization for 10,000+ row tables.
5. **ClearMessage is unscoped.** Any client sending `ClearMessage` clears all frames and scenes for all clients. No per-client or per-frame scoping.
6. **No frame-scoped events.** `InteractionMessage` carries `element_id` but not `frame_id`. Events from elements in different frames with the same element ID are indistinguishable.

9 Security

9.1 Threat Model

The primary trust boundary is the Unix socket. The display server accepts connections from any process that can reach the socket file. In the default configuration (`/tmp/lux-$USER/`), the socket is protected by filesystem permissions (user-only directory).

Threat	Description	Mitigation
Socket hijacking	Another process connects to the socket and injects scenes or reads events.	Directory permissions (0700) on `/tmp/lux-$USER/`.
Malformed messages	Client sends invalid JSON, unknown message type, or oversize payload.	Disconnects client; no crash. Oversize payloads rejected by FrameReader.
Render function	Agent-submitted Python code executes arbitrary operations.	Consent dialog with AST warnings. User must explicitly click Allow.
Resource exhaustion	Scene with millions of elements or very large images.	16 MiB message cap. No element count limit (ImGui handles gracefully).

9.2 Render Function Security

Render functions are the only element kind that executes arbitrary code. The security model is **consent-based, not sandbox-based**:

1. **AST scanning** (`src/punt_lux/ast_check.py`): a best-effort visitor flags suspicious imports (`os`, `subprocess`, `socket`, `ctypes`, `urllib`, etc.) and dangerous builtins (`eval`, `exec`, `open`, `__import__`). This is a *UX signal*, not a security boundary. It can be trivially bypassed (e.g., `getattr(__builtins__, "eval")`).

2. **Consent dialog** (`src/punt_lux/consent.py`): an ImGui modal displays the full source code with line numbers and any AST warnings. The user must click "Allow" to proceed. Dismissing the modal (Esc) is treated as "Deny." Consent is per-element: changing the source code resets consent.

3. **Execution** (`src/punt_lux/runtime.py`):
   `CodeExecutor` compiles the source and calls `render(ctx)` each frame. The `RenderContext` provides a persistent state dict, delta time, frame counter, canvas dimensions, and a `send()` callback for emitting events. Execution is **not sandboxed**: the code runs in the display server's process with full access to the Python runtime. Runtime errors are caught and displayed, not propagated.

Implication: render functions should be treated as user-approved code, equivalent to running a script the user downloaded. The consent dialog provides informed consent, not isolation.

9.3 Protocol Robustness

- **Malformed JSON**: client disconnected, not crashed. The display logs the error and removes the client from `_clients`.
- **Unknown message type**: client disconnected.
- **Oversize frame**: `FrameReader` checks `bufSize` $\leq$ `maxBufSize` and disconnects on violation.
- **Partial reads**: `FrameReader` accumulates bytes across frames; no assumption of complete messages per `recv()`.
- **Identity fields protected**: `_apply_update()` silently drops patches that attempt to change `id` or `kind` on an element.
- **Double-remove idempotent**: `_remove_client()` is safe to call twice on the same socket.

10 Platform Integration

10.1 macOS

- **Dock hiding.** After GLFW initialization (in `_on_post_init()`), the display server sets `NSApplicationActivationPolicyAccessory` on the shared `NSApplication`. GLFW's `glfwInit()` sets the policy to Regular; the post-init call overrides it. The Dock icon briefly flashes at launch. Requires `pyobjc-framework-Cocoa` (optional `display` extra).
- **Process name.** `setproctitle("Lux")` makes the process identifiable in `ps` and `top`. Does not affect Activity Monitor's binary name column.
- **Font discovery.** Prefers Arial Unicode or Helvetica; merges Apple Symbols for mathematical/Z notation glyphs.
- **AF_UNIX path limit.** macOS limits Unix socket paths to ~104 characters. `tempfile.mkdtemp(prefix="lux-")` used in tests; production paths are short by design.

10.2 Linux

- **Font discovery.** DejaVu Sans or Noto Sans; merges Noto Sans Symbols2 for Unicode coverage.
- **XDG compliance.** Prefers `$XDG_RUNTIME_DIR/lux/` for socket placement.
- **No Dock equivalent.** Window managers use the X11 window title (set to "Lux").

11 MCP Tool Surface

Tool	Signature and Purpose
`show`	`(scene_id, elements, title?, layout?, frame_id?, frame_title?, frame_size?, frame_flags?, frame_layout?)` — Display a scene. Comprehensive docstring lists all 27 element kinds with examples. Frame parameters enable tab-isolated windows.
`show_table`	`(scene_id, columns, rows, filters?, detail?, flags?, title?, frame_id?, frame_title?)` — Convenience wrapper for filterable data tables with search, combo filters, and detail panel. Delegates to `show`.
`show_dashboard`	`(scene_id, metrics?, charts?, table_columns?, table_rows?, title?)` — Metrics cards + charts + summary table. Delegates to `show`.
`show_diagram`	`(scene_id, layers, edges?, title?)` — Layered architecture diagram with auto-layout, routed edges, and arrowheads. Delegates to `show`.
`update`	`(scene_id, patches)` — Incremental JSON patches targeting elements by ID.
`clear`	`()` — Clear all scenes.
`ping`	`()` — Latency probe; returns `PongMessage`.
`recv`	`(timeout?)` — Receive next event (interaction, window lifecycle).
`set_menu`	`(menus)` — Set agent-provided menu bar entries.
`register_tool`	`(label, tool_id)` — Register a World menu item routed only to the calling server via `recv()`.
`set_theme`	`(theme)` — Apply display theme.
`display_mode`	`(mode?)` — Get or set display mode (y/n); advisory L3 signal for consumer plugins.

12 Formal Model

The display server's state machine is formally specified in `docs/display-server.tex` using the Z notation (ISO 13568). The specification covers:

- Client connection/disconnection with FrameReader association.
- Scene replacement, incremental patching, and clearing.
- Interaction event generation and broadcast.
- FrameReader streaming buffer invariants.

The specification has been type-checked with Spivey's `fuzz` and model-checked with ProB (7,943 reachable states explored). Refinement tests in `tests/test_display_refinement.py` verify commutativity between the abstract Z operations and the concrete Python implementation via an abstraction function (`tests/display_abstraction.py`).

Partition tests in `tests/test_display_partition.py` derive test cases from the Z specification using the Test Template Framework (TTF), covering all input space partitions of each operation schema.

Note: the formal model predates frames and models a single-scene display. The abstraction function maps multi-scene state to the active tab for backward compatibility. A specification update to model frames, shared ownership, and orphan semantics is pending.

13 Test Architecture

Test File	Count	What It Tests
`test_protocol.py`	162	Serialization, deserialization, framing
`test_display_partition.py`	107	TTF-derived partitions from Z spec
`test_server.py`	70	MCP tool behavior (mocked client)
`test_display_state.py`	65	State machine logic (no ImGui)
`test_display_refinement.py`	28	Abstraction function commutativity
`test_client.py`	26	Client library (real sockets, threads)
`test_runtime.py`	23	CodeExecutor lifecycle
`test_show.py`	20	CLI show subcommands
`test_hooks.py`	17	Hook handler output
`test_paths.py`	16	Socket discovery, PID, auto-spawn
`test_config.py`	16	Config read/write
`test_ast_check.py`	12	AST visitor coverage
`test_cli.py`	11	CLI command smoke tests
`test_consent.py`	9	Consent dialog state machine
`test_apps_clock.py`	7	Clock applet rendering
`test_unit.py`	6	Misc unit tests
`test_apps_calculator.py`	3	Calculator applet logic
`test_version.py`	2	Version string consistency
Total	629	(11 e2e/integration deselected in CI)

Quality gates: `ruff` (lint + format), `mypy` (strict), `pyright` (strict), `pytest` (all markers except slow/integration/e2e). Zero violations required.